EcoColeta API

Pipeline CI/CD, Containerização e Deploy Multiambiente

Desafio “Navegando pelo mundo DevOps” — projeto ESG “Gestão de Resíduos e Reciclagem”, construído sobre a API da Fase 4.

Integrantes	Gabriel Rodrigues de Lima Furquim de Moraes
	Raphael Henrique Soares Semede
	Crystian Lopes do Carmo
	Jefferson Nery Martins
Curso / Turma	FIAP — 2TDS

Projeto base	EcoColeta API (ASP.NET Core 8 + SQL Server)
Ferramenta de CI/CD	GitHub Actions
Orquestração (local)	Docker Compose (API + SQL Server)
Registro de imagem	GitHub Container Registry (GHCR)
Infraestrutura de deploy	Azure App Service + Azure SQL Database (Azure for Students)

1. Descrição do pipeline

O pipeline é implementado em **GitHub Actions** (`.github/workflows/ci-cd.yml`) e roda em quatro jobs encadeados, cada um só avançando se o anterior terminar com sucesso.

Job	Gatilho	O que faz
build-and-test	push, pull request, tag ou manual	dotnet restore -> build -> test (xUnit). Publica o .trx como artefato do run.
docker-build-push	idem, exceto em pull request	Builda a imagem a partir do Dockerfile e publica no GHCR com tags sha-<commit>, staging (push na main) e production + vX.Y.Z (tag de versão).
deploy-staging	push na branch main	Aponta o Azure Web App ecocoleta-staging para a imagem sha-<commit> dessa build no GHCR (azure/webapps-deploy). Health check contra /health antes de concluir.
deploy-production	push de uma tag vX.Y.Z	Mesmo deploy, Web App ecocoleta-producao, imagem sha-<commit>. Fica retido aguardando aprovação manual do Environment “production” antes de rodar.

Por que staging é automático e produção não

Os dois últimos jobs usam os **Environments** do GitHub (Settings -> Environments). O Environment `production` tem um “Required reviewer” configurado — o pipeline chega até ele, mostra o deploy pendente na aba Actions, e só executa depois de um clique de aprovação manual. O Environment `staging` não tem essa trava, permitindo deploy contínuo a cada push na main.

Para onde os deploys apontam: Azure App Service

Os jobs `deploy-staging` e `deploy-production` rodam em runner hospedado (`ubuntu-latest`) e usam a action `azure/webapps-deploy` para apontar cada **Azure Web App** (App Service para Containers) para a imagem correspondente no GHCR, autenticando com o publish profile do Web App (Secret do repositório). Não há servidor ou VM próprios para manter — o Azure é quem executa o container.

Staging e produção são dois Web Apps separados (`ecocoleta-staging` / `ecocoleta-producao`), cada um com seu próprio **Azure SQL Database** — mesmo provider EF Core já usado no projeto (`Microsoft.EntityFrameworkCore.SqlServer`), sem alterar código. Os recursos são criados uma única vez com `scripts/azure-setup.sh`, usando o crédito da conta **Azure for Students**.

Por que a tag de deploy é sha-<commit> e não :staging/:production

O Azure App Service só reinicia o container quando a *referência* da imagem muda. Reutilizar sempre o mesmo nome de tag (ex: `:staging`) fazia o Azure assumir que nada havia mudado entre deploys e nunca puxar a imagem nova — o job terminava com sucesso no GitHub Actions, mas o container antigo continuava rodando. A correção foi apontar o deploy para a tag imutável gerada a cada build (`sha-<commit>`), garantindo que cada deploy sempre reflita o commit publicado.

2. Docker: arquitetura

A API roda em container, conectada a um SQL Server também em container — substituindo o SQL Server LocalDB usado na Fase 4, que é específico do Windows e não existe dentro de um container Linux. Essa dupla (API + SQL Server) via Docker Compose é o ambiente local/de desenvolvimento; em staging e produção na nuvem, a mesma imagem da API roda no Azure App Service, conectada a um Azure SQL Database (ver seção 1).

Dockerfile — build multi-stage

Três estágios: **build** (SDK .NET 8, restaura e compila), **publish** (gera o artefato de publicação) e **final** (runtime ASP.NET, bem mais leve que o SDK). Só o resultado do publish entra na imagem final.

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["Fiap.EcoColeta.Api.csproj", "./"]
RUN dotnet restore "Fiap.EcoColeta.Api.csproj"
COPY . .
RUN dotnet build "Fiap.EcoColeta.Api.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "Fiap.EcoColeta.Api.csproj" -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
EXPOSE 8080
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "Fiap.EcoColeta.Api.dll"]
```

Orquestração — docker-compose.yml + overlays

Um arquivo base define o que é comum aos dois ambientes (serviços `api` e `db`, rede, variáveis parametrizadas, health check do banco). Dois overlays definem só o que muda: nome dos containers, porta publicada e volume do banco.

Arquivo	Papel
docker-compose.yml	Serviços api + db, rede ecocoleta-net, variáveis via ${...}, health check do SQL Server (sqlcmd)
docker-compose.staging.yml	Container *-staging, porta 8081:8080, volume ecocoleta-db-staging-data
docker-compose.production.yml	Container *-production, porta 8080:8080, volume ecocoleta-db-production-data
.env.staging / .env.production	SA_PASSWORD, ASPNETCORE_ENVIRONMENT, JWT_ISSUER, JWT_KEY — nunca versionados

Comandos usados

```
# build + validação da imagem
docker build -t ecocoleta-api -f Fiap.EcoColeta.Api/Dockerfile Fiap.EcoColeta.Api

# subir staging (porta 8081)
docker compose -f docker-compose.yml -f docker-compose.staging.yml \
--env-file .env.staging up -d --build

# subir produção (porta 8080)
docker compose -f docker-compose.yml -f docker-compose.production.yml \
--env-file .env.production up -d --build
```

3. Desafios encontrados e como foram resolvidos

Colocar o pipeline pra rodar de verdade contra a infraestrutura do Azure (em vez de só funcionar “no papel”) expôs problemas que só aparecem com a infra real — e foram a parte mais instrutiva do desafio.

Região bloqueada por política da subscription (Azure for Students)

Tentativas de criar o App Service Plan e o SQL Server em eastus/brazilsouth falhavam com RequestDisallowedByAzure — a subscription de testes só libera algumas regiões específicas (chilecentral funcionou). Resolvido fixando a região no script e passando --location explícito em todo az ... create: sem essa flag, o recurso tentava herdar a região do resource group e falhava mesmo com a variável correta definida.

Quota de 1 App Service Plan por subscription

Mesmo após corrigir a região, a criação de um segundo App Service Plan falhava — contas Azure for Students limitam a subscription a um único plano. Resolvido detectando, no script, um plano Linux já existente (reserved==true) e reaproveitando-o em vez de tentar criar outro.

azure/webapps-deploy falhando com “Failed to get app runtime OS”

Web Apps novos no Azure vêm com a autenticação básica do SCM/Kudu desativada por padrão — e é justamente essa autenticação que o publish profile usa para aplicar o deploy. Resolvido com az resource update ... basicPublishingCredentialsPolicies ... properties.allow=true em cada Web App, automatizado no script de setup.

Deploy “verde” no Actions, mas o container nunca atualizava

Ver a nota na seção 1: reutilizar a mesma tag de imagem a cada deploy não fazia o Azure perceber que havia uma imagem nova. Resolvido trocando para a tag imutável sha-<commit>, gerada a cada build.

Health check de produção sempre expirava

O Program.cs desativa o Swagger de propósito em Production (boa prática — não expor documentação interativa em produção), mas o health check do pipeline testava justamente /swagger/index.html, que retorna 404 nesse ambiente. Resolvido com um endpoint dedicado /health, disponível em qualquer ambiente.

Login retornando 500 genérico em produção

A chave JWT configurada no Web App tinha 184 bits — o algoritmo HS256 exige no mínimo 256 bits, e a exceção (IDX10720) ficava escondida atrás do middleware de tratamento de erros, retornando só uma mensagem genérica ao cliente. Resolvido gerando a chave com openssl rand -base64 32 (256 bits) e atualizando o app setting Jwt__Key nos dois Web Apps; o diagnóstico só foi possível olhando o log do container em tempo real (az webapp log tail).

LocalDB não existe em container Linux

A Fase 4 rodava contra SQL Server LocalDB (processo nativo do Windows). Resolvido trocando o destino, nos ambientes de staging/produção, para um container oficial do SQL Server 2022, com a connection string injetada por variável de ambiente (ConnectionStrings__DatabaseConnection) — sem alterar o fluxo local em Development, que continua usando LocalDB normalmente.

Condição de corrida na subida do banco

Mesmo com o health check do Compose (service_healthy), a API podia tentar conectar no instante exato em que o SQL Server ainda estava finalizando a inicialização interna. Resolvido adicionando, no Program.cs, uma rotina de retry (até 8 tentativas, com espera crescente) em torno da aplicação das migrações no startup.

Isolar staging de produção na mesma máquina

Resolvido com overlays de Compose separados: cada ambiente tem seu próprio nome de container, porta publicada e volume nomeado do banco, então os dois podem rodar ao mesmo tempo sem conflito nem risco de staging sobrescrever dados de produção.

Separar o que exige aprovação do que não exige

Resolvido usando os Environments do GitHub: production tem um required reviewer configurado, então o pipeline fica retido até alguém aprovar manualmente; staging não tem essa trava e recebe deploy contínuo.

Onde rodar o deploy sem manter servidor próprio

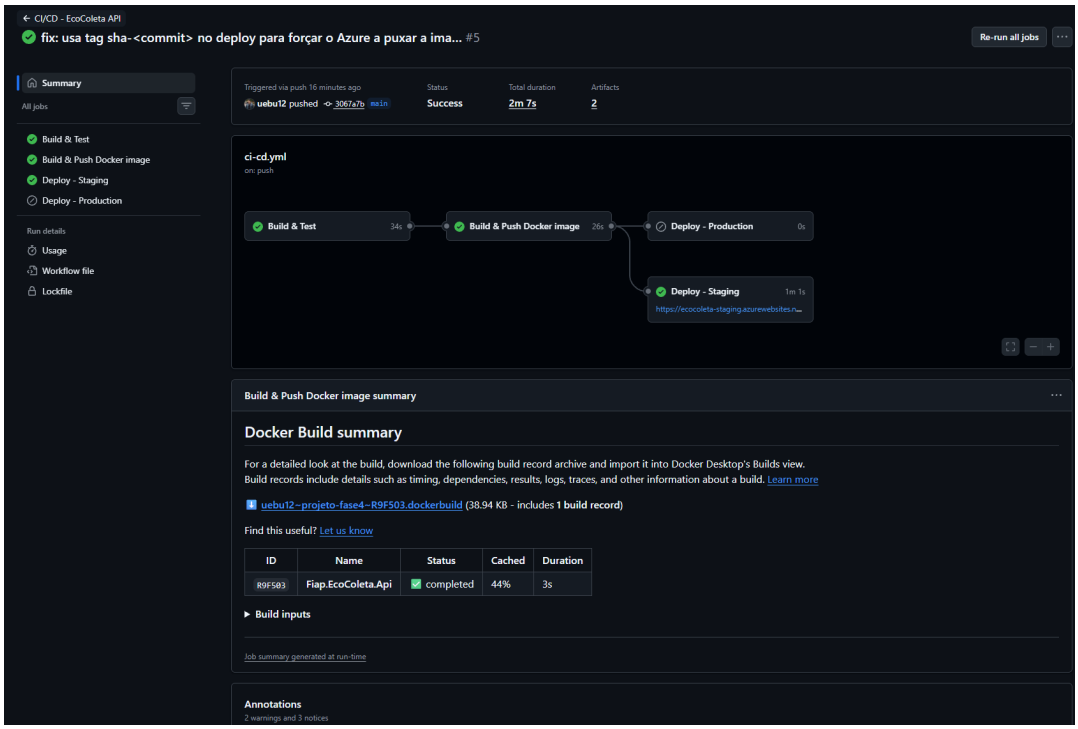
Deploy automatizado em dois ambientes pedia um destino real na nuvem, não só o docker-compose.yml local. Resolvido usando Azure App Service (Web App for Containers): o Azure passa a executar a imagem publicada no GHCR diretamente, sem precisar provisionar ou manter VM, usando o crédito da conta Azure for Students.

Trocar o banco sem reescrever a camada de dados

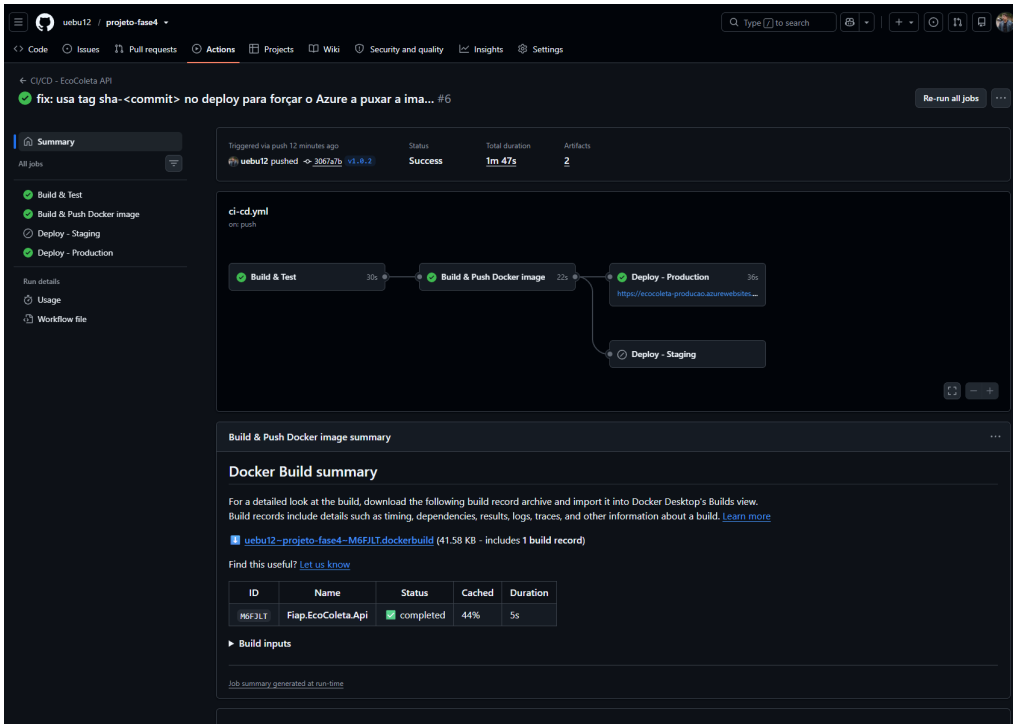
Staging/produção precisavam de um banco real na nuvem, não o SQL Server LocalDB da Fase 4. Resolvido com Azure SQL Database — compatível com o mesmo provider Microsoft.EntityFrameworkCore.SqlServer já usado no projeto, então a troca foi só de connection string (variável de ambiente no Web App), sem alterar Models, Repository ou migrações.

4. Prints do pipeline e dos ambientes

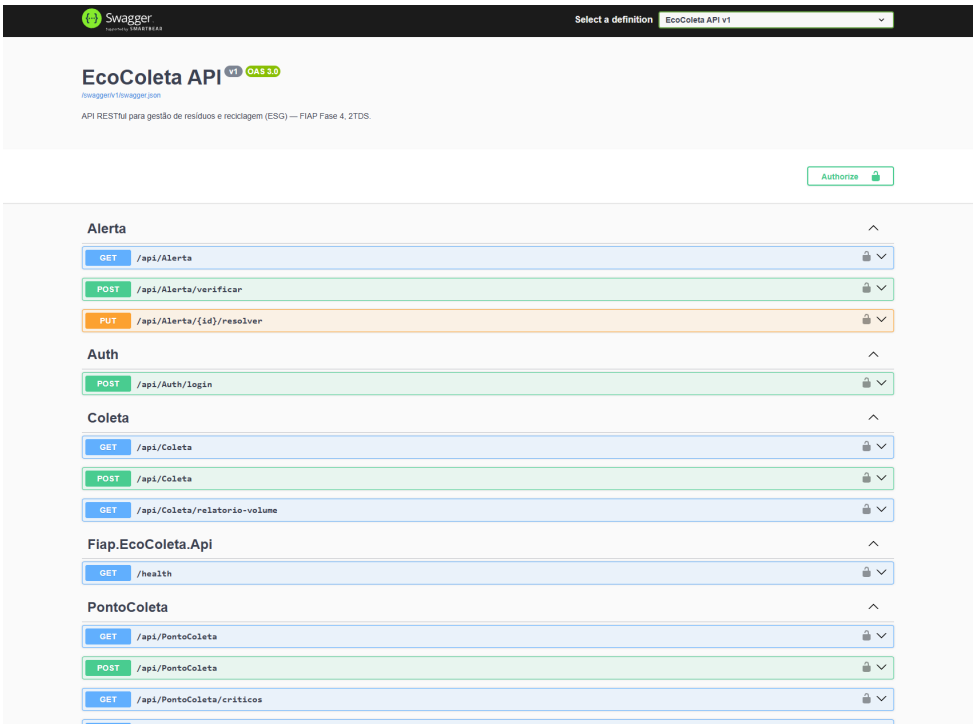
Capturas reais, tiradas rodando o pipeline de ponta a ponta contra os dois ambientes no Azure (não simulação).



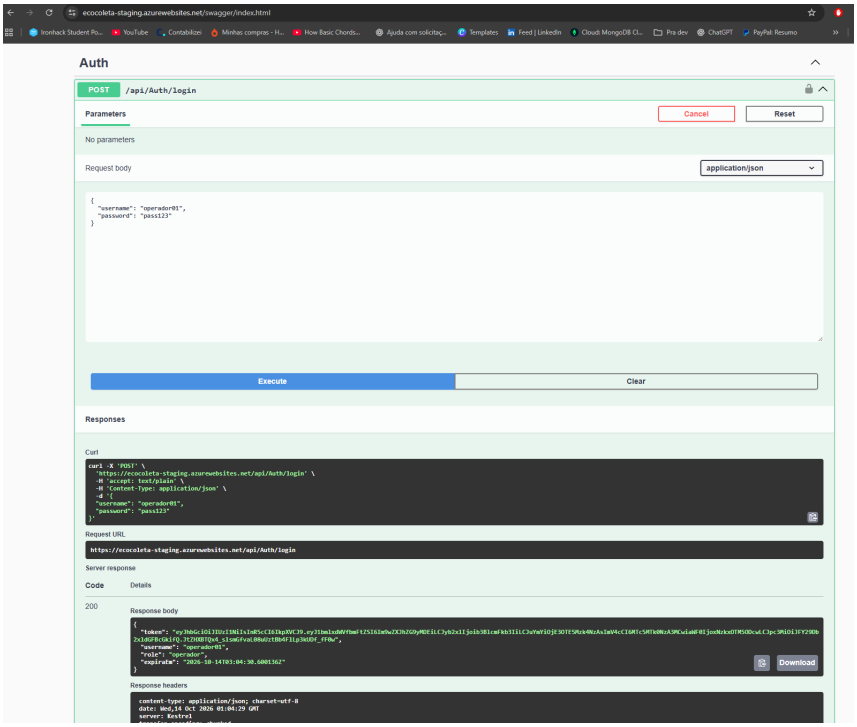
Pipeline no GitHub Actions — build-and-test, docker-build-push e deploy-staging concluídos



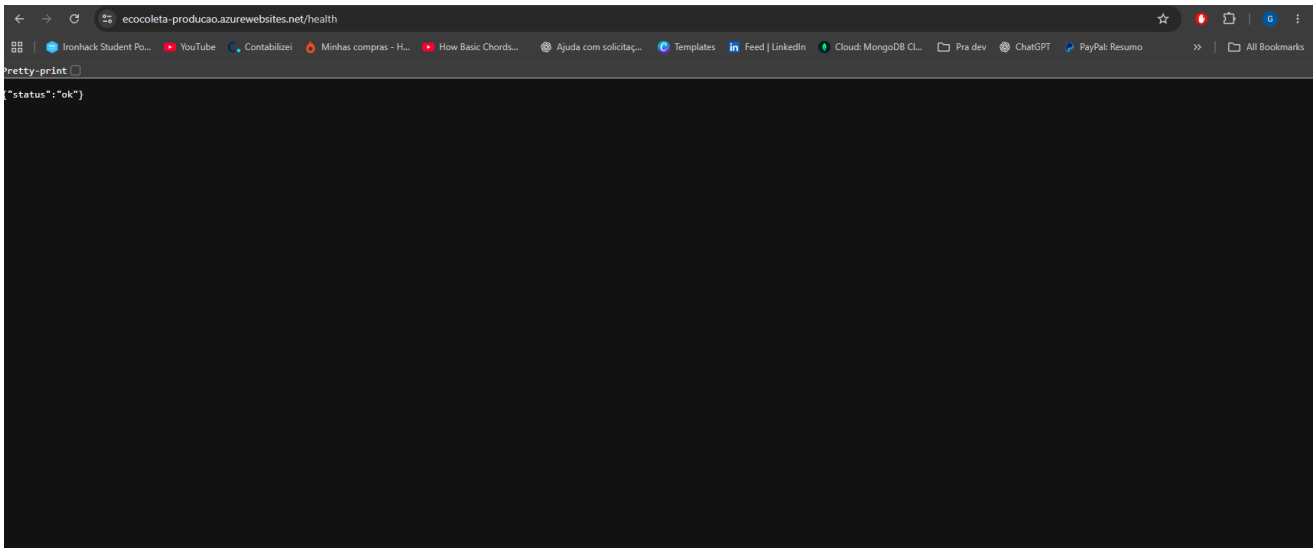
deploy-production concluído, após aprovação manual do Environment “production”



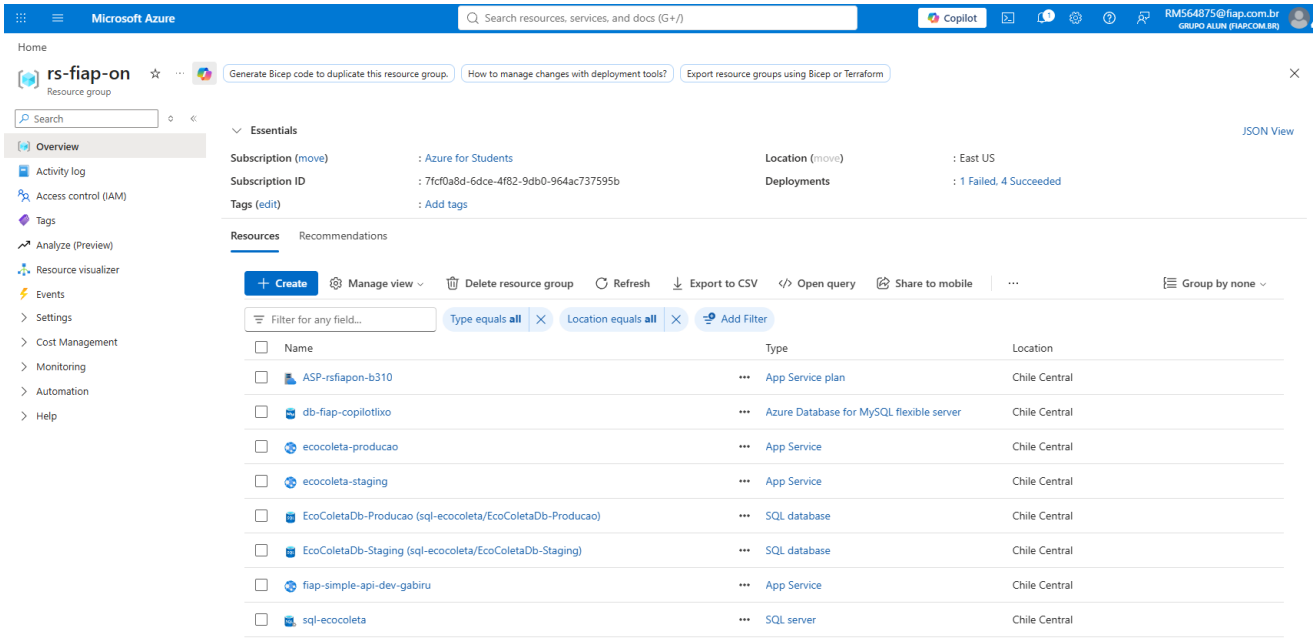
Swagger respondendo em staging, com o endpoint /health listado junto dos demais



Login via Swagger contra staging, retornando o token JWT



https://ecocoleta-producao.azurewebsites.net/health respondendo {"status":"ok"} (produção não expõe Swagger de propósito)



Recursos criados no Azure: resource group rs-fiap-on com os dois Web Apps, o SQL Server lógico e os dois bancos (um por ambiente)

5. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	OK
Dockerfile funcional	OK
docker-compose.yml ou arquivos Kubernetes	OK
Pipeline com etapas de build, teste e deploy	OK
README.md com instruções e prints	OK
Documentação técnica com evidências (PDF ou PPT)	OK
Deploy realizado nos ambientes staging e produção	OK